

# NLP Course Project

# HUST

**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.



ĐẠI HỌC  
BÁCH KHOA HÀ NỘI  
HANOI UNIVERSITY  
OF SCIENCE AND TECHNOLOGY

# MULTITASK BERT FOR STUDENT SUPPORT UNDERSTANDING

A self-implemented BERT encoder for SST, paraphrase detection, and STS

**Người thực hiện:**

Vu Thuong Tin

Chu Anh Duc

Tran Quang Hung

Nguyen Xuan Khai

Do Dang Vu

ONE LOVE. ONE FUTURE.

School of Information and Communication Technology – HUST

1. Problem Statement
2. Approach
3. Methodology
4. Training
5. Results
6. Demo

# 1. Problem Statement

Student-support systems often receive short natural-language requests that require different types of understanding.

## Core question

Can one shared BERT encoder support sentiment classification, duplicate-question detection, and semantic similarity at the same time?

- ▶ **Sentiment:** detect the tone of a student message.
- ▶ **Paraphrase:** detect duplicate or equivalent questions.
- ▶ **Similarity:** retrieve semantically related handbook entries.

## 2. Tasks and Datasets

| Task                     | Dataset              | Metric              |
|--------------------------|----------------------|---------------------|
| Sentiment classification | SST-5                | Accuracy            |
| Paraphrase detection     | Quora Question Pairs | Accuracy            |
| Semantic similarity      | STS-B                | Pearson correlation |

The three tasks are different in output format, but all depend on contextual sentence representations.

### 3. Proposed Approach

---

#### Shared representation

- ▶ One BERT encoder.
- ▶ Three task-specific heads.
- ▶ Joint multitask fine-tuning.

The HuggingFace backend is used only as a sanity-check reference.

#### Main contribution

We re-implemented the BERT encoder ourselves instead of using `AutoModel` in the main forward path.

## 4. Why Multitask Learning?

- ▶ SST learns single-sentence sentiment.
- ▶ Quora learns binary semantic equivalence.
- ▶ STS learns graded semantic similarity.
- ▶ Para and STS share sentence-pair structure.

### Hypothesis

Sharing the encoder and relational sentence-pair features can improve representation learning across related tasks.

## 5. BERT Encoder Architecture

### Core blocks

- ▶ Token ids
- ▶ Token embeddings
- ▶ Position embeddings
- ▶ Segment embeddings
- ▶ Self-attention
- ▶ Feed-forward
- ▶ Residual + LayerNorm

### Diagram placeholder

- ▶ Insert BERT diagram.
- ▶ Input → embeddings.
- ▶ Encoder stack.
- ▶ [CLS] output.

## 6. Self-Implementation Validation

### Parity check against HuggingFace

Same input and same pretrained weights:

- ▶ Max hidden-state difference:  $4.29 \times 10^{-6}$ .
- ▶ Max pooler-output difference:  $9.54 \times 10^{-7}$ .

This validates that the custom encoder is numerically faithful before fine-tuning.

## 7. Multitask Architecture

| Component           | Role                      | Output            |
|---------------------|---------------------------|-------------------|
| Shared BERT encoder | Contextual representation | [CLS] vector      |
| SST head            | Sentiment classifier      | 5 logits          |
| Paraphrase head     | Duplicate detector        | 1 logit           |
| STS head            | Similarity regressor      | score in $[0, 5]$ |
| Relational layer    | Shared Para–STS features  | pair interaction  |

The architecture separates task-specific decisions while keeping the language representation shared.

## 8. Rich Relational Layer

Para and STS are both sentence-pair tasks. We therefore share an interaction representation before branching into task-specific heads.

$$z = \frac{h_u \odot h_v}{\|h_u\|_2 \|h_v\|_2 + \epsilon}$$

- ▶  $h_u, h_v$ : pooled sentence vectors.
- ▶ Element-wise interaction keeps dimension-level information.
- ▶ STS can benefit from the larger Quora signal.

# 9. Training Pipeline

## Pipeline

- ▶ Load tokenizer
- ▶ Load BERT weights
- ▶ Build dataloaders
- ▶ Interleaved batches
- ▶ Task losses
- ▶ Save best checkpoint

## Diagram placeholder

- ▶ Insert training flow.
- ▶ SST / Para / STS.
- ▶ Shared encoder.
- ▶ Task heads.
- ▶ Aggregate score.

## Best recipe

- ▶ Optimizer: AdamW.
- ▶ Learning rate:  $2e^{-5}$ .
- ▶ Weight decay: 0.01.
- ▶ Epochs: 20.
- ▶ Schedule: linear warmup/decay.

Checkpoints are selected by aggregate development score.

# 11. Evaluation Protocol

Each task is evaluated with its standard metric:

- ▶ SST: accuracy.
- ▶ Paraphrase: accuracy.
- ▶ STS: Pearson correlation.

Aggregate score:

$$S = \frac{SST_{acc} + Para_{acc} + (STS_r + 1)/2}{3}$$

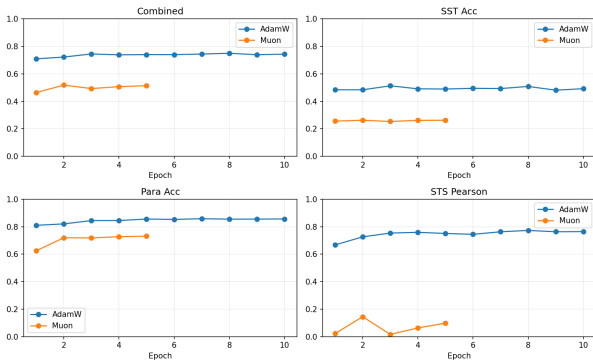
## 12. Main Results

| Model                       | SST          | Para         | STS          | Score        |
|-----------------------------|--------------|--------------|--------------|--------------|
| Self-impl AdamW linear 20e  | 0.520        | <b>0.864</b> | 0.785        | <b>0.759</b> |
| HF sanity baseline 10e      | <b>0.526</b> | 0.861        | <b>0.790</b> | 0.761        |
| Self-impl AdamW constant LR | 0.505        | 0.860        | 0.784        | 0.752        |
| Self-impl Muon safe 20e     | 0.523        | 0.851        | 0.750        | 0.750        |
| Muon high-LR early run      | 0.262        | 0.720        | 0.145        | 0.518        |

Our self-implemented encoder reaches 0.759, nearly matching the library-backed baseline at 0.761.

# 13. Optimizer Finding

Report-best AdamW vs Muon training history



## Conclusion

- ▶ AdamW is clearly better.
- ▶ Muon damages pretrained weights.
- ▶ Para / STS drop sharply.
- ▶ Final choice: AdamW.

# 14. Error Analysis

## Current weakness

STS is the weakest task: Pearson  $r = 0.785$ .

## To complete

Insert 2–3 real STS error examples:

- ▶ sentence A / sentence B,
- ▶ gold score / predicted score,
- ▶ short error explanation.

# 15. Demo A: Checkpoint Inference

## Purpose

Directly load one checkpoint and test:

- ▶ SST sentiment.
- ▶ Paraphrase probability.
- ▶ STS similarity score.

`scripts/demo_gradio.py`

### **Screenshot Placeholder**

Replace with checkpoint inference  
screenshot.

## 16. Demo B: Smart Student Support

### Product workflow

- ▶ Detect student sentiment.
- ▶ Match duplicate FAQs.
- ▶ Rank handbook entries.

`scripts/demo_student_support.py`

### Screenshot Placeholder

Replace with Smart Student Support  
screenshot.

### Checkpoint inference

```
PYTHONPATH=src python -m scripts.demo_gradio \  
-config configs/report_best_self_impl_20e_linear_warmup_lr2e5.yaml \  
-checkpoint runs/report_best_self_impl_20e_linear_warmup_lr2e5/best.pt
```

### Smart student support

```
PYTHONPATH=src python -m scripts.demo_student_support \  
-config configs/report_best_self_impl_20e_linear_warmup_lr2e5.yaml \  
-checkpoint runs/report_best_self_impl_20e_linear_warmup_lr2e5/best.pt
```

## 18. Conclusions

- ▶ We re-implemented the BERT encoder.
- ▶ Encoder parity is extremely close:  $4.29 \times 10^{-6}$ .
- ▶ Self-implemented score: **0.759**.
- ▶ HF baseline score: 0.761.
- ▶ AdamW clearly outperforms Muon.
- ▶ Two demos connect benchmarks to usage.

Future work: stronger STS calibration, dynamic task weighting, and richer sentence-pair modeling.

# THANK YOU

**Project:** Multitask BERT for Student Support Understanding

**Repository:** multitask-bert-v2

Q&A

The logo for HUST (Ho Chi Minh University of Science) is displayed in white, bold, sans-serif capital letters. It is centered within a dark blue rectangular area. Surrounding the text is a decorative pattern of red dots of varying sizes, arranged in concentric, slightly irregular circles, creating a textured, circular effect.

- [1] Devlin et al., *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, 2019.
- [2] Liu et al., *Multi-Task Deep Neural Networks for Natural Language Understanding*, 2019.
- [3] Reimers and Gurevych, *Sentence-BERT*, 2019.
- [4] Loshchilov and Hutter, *Decoupled Weight Decay Regularization*, 2019.